

NOOR

Documentation Technique & Fonctionnelle

Application Mobile Islamique

Version	3.4.0 — Aesthetic Edition
Plateforme	iOS · Android · Web
Framework	React Native / Expo SDK 51
Langage	TypeScript 5.3
Date	Juin 2026
Auteur	Safia Massaid

100% Local · Aucune donnée envoyée · Open Source

TABLE DES MATIÈRES

1. Présentation du projet
2. Architecture technique
3. Structure des fichiers
4. Navigation & écrans
5. Écrans détaillés
 - 5.1 *Accueil (HomeScreen)*
 - 5.2 *Découvrir (DiscoverScreen)*
 - 5.3 *Statistiques (StatsScreen)*
 - 5.4 *Paramètres (SettingsScreen)*
 - 5.5 *Onboarding*
6. Composants réutilisables
7. Système de thème Dark/Light
8. Algorithme des horaires de prière
9. Algorithme des streaks
10. Flux de données & persistance
11. Compatibilité Web
12. Lancer le projet

1. PRÉSENTATION DU PROJET

Noor (■■■ — « lumière » en arabe) est une application mobile islamique de suivi des habitudes spirituelles quotidiennes. Elle accompagne le musulman dans sa pratique en combinant plusieurs fonctionnalités au sein d'une interface élégante, entièrement locale et sans aucune collecte de données.

■	Suivi des habitudes	Cocher ses actes spirituels du jour
■	Horaires de prière	Calcul automatique par GPS, 3 méthodes
■	Statistiques	Streaks, calendrier mensuel, progression par habitude
■	Boussole Qibla	Orientation vers La Mecque
■	Citadelle du Musulman	14 du'as authentiques avec traduction
■	Mode sombre automatique	S'adapte à l'heure du Maghrib
■	Notifications	Rappels de prières et d'habitudes personnalisés

Philosophie de design

L'interface suit une esthétique « **pro white premium** » : tons ivoire, accents dorés, typographie élégante, ombres douces. Le mode sombre utilise un vert nuit profond (inspiration des mosquées nocturnes). L'ensemble vise la sobriété et la sérénité, en cohérence avec la dimension spirituelle de l'application.

2. ARCHITECTURE TECHNIQUE

Stack technologique

Couche	Technologie	Version
Framework	React Native	0.74.5
SDK	Expo	51.0.0
Navigation	React Navigation (Bottom Tabs)	6.x
Persistance	AsyncStorage (local)	—
Graphiques	react-native-svg	15.3.0
Icons	@expo/vector-icons (Ionicons)	—
Notifications	expo-notifications	0.28
Géolocalisation	expo-location	17.0
Typage	TypeScript	5.3
Bundler web	Metro (via Expo)	—

Principes d'architecture

- Architecture **100% locale** : aucune donnée n'est envoyée vers un serveur externe.
- **ThemeProvider** (Context React) : propagation du thème dark/light à tous les composants.
- **NavigationContainer** avec Bottom Tabs : 4 onglets principaux (Accueil, Découvrir, Stats, Réglages).
- **AsyncStorage** : persistance clé/valeur chiffrée sur l'appareil pour logs, habitudes et paramètres.
- Stubs Metro pour la **compatibilité web** : remplacement des modules natifs par des implémentations légères.

3. STRUCTURE DES FICHIERS

L'application représente environ **4 300 lignes** de code TypeScript/JavaScript réparties comme suit :

Fichier / Dossier	Rôle
App.tsx	Entrée — ThemeProvider + Navigation + Onboarding
src/screens/HomeScreen.tsx	Accueil : horaires de prière + habitudes du jour
src/screens/DiscoverScreen.tsx	Boussole Qibla + bibliothèque de du'as
src/screens/StatsScreen.tsx	Statistiques : streak, calendrier, progression
src/screens/SettingsScreen.tsx	Paramètres : prières, habitudes, thème
src/screens/OnboardingScreen.tsx	Onboarding 3 slides (premier lancement)
src/components/HabitCard.tsx	Carte d'une habitude avec toggle animé
src/components/ProgressRing.tsx	Anneau SVG de progression
src/components/MonthCalendar.tsx	Calendrier mensuel avec heatmap
src/components/Confetti.tsx	Animation confetti (objectif atteint)
src/contexts/ThemeContext.tsx	Context React pour le thème global
src/data/habits.ts	Habitudes par défaut + interface Habit
src/data/duas.ts	14 du'as authentiques avec catégories
src/constants/theme.ts	LIGHT_COLORS, DARK_COLORS, SPACING, RADIUS
src/constants/utils.ts	Fonctions utilitaires + AsyncStorage
src/utils/prayerCalc.ts	Algorithme astronomique de calcul des prières
src/stubs/	Modules natifs simulés pour le web

4. NAVIGATION & ÉCRANS

L'application utilise **React Navigation 6** avec une barre d'onglets inférieure (Bottom Tabs). Au premier lancement, l'utilisateur passe par trois slides d'onboarding avant d'accéder aux onglets principaux.

Onglet	Écran principal	Fonctionnalités clés
■ Accueil	HomeScreen	Horaires de prière, habitudes du jour, verset
■ Découvrir	DiscoverScreen	Boussole Qibla, du'as (Citadelle du Musulman)
■ Stats	StatsScreen	Streak, heatmap calendrier, stats par habitude

■■■ Réglages	SettingsScreen	Prières, thème, habitudes perso, profil
--------------	----------------	---

5. ÉCRANS DÉTAILLÉS

5.1 Écran d'accueil — HomeScreen

Hub central de l'application. L'utilisateur y consulte les horaires de prière et coche ses habitudes du jour.

- **Stagger animation** : les cartes apparaissent en cascade (60 ms entre chaque).
- **Confetti** : déclenché quand toutes les habitudes sont cochées.
- **Countdown** de la prochaine prière mis à jour chaque minute.
- **Habitudes custom** chargées via `getAllHabits()` à chaque focus.
- **Refresh automatique** à minuit pour réinitialiser la journée.
- Affichage de la **date grégorienne et hégirique** (Hijri).

5.2 Écran Découvrir — DiscoverScreen

Outils spirituels additionnels — orientation Qibla et bibliothèque de du'as.

- **Boussole interactive** : ring SVG avec ticks tous les 5°, labels N/E/S/O.
- Drag horizontal pour faire tourner le ring → angle mis à jour en temps réel.
- **Badge « Aligné ✓ »** vert quand l'angle est dans $\pm 6^\circ$ de la Qibla.
- **14 du'as** authentiques avec source (Bukhari, Muslim, Abu Dawud...).
- **Filtre par catégorie** : Matin · Soir · Prière · Manger · Voyage · Masjid.
- **Modal bottom sheet** : texte arabe complet + traduction + source.

5.3 Écran Statistiques — StatsScreen

Visualisation de la progression spirituelle sur 30 jours.

- **Streak** : nombre de jours consécutifs avec au moins 1 habitude cochée.
- **Heatmap** : calendrier avec 4 niveaux de couleur selon le % du jour.
- **3 boîtes de stats** : jours actifs, total d'habitudes, moyenne en %.
- **Stats par habitude** : respecte les `targetDays` (ex: jeûne lun/jeu uniquement).
- **Barre hebdomadaire** : progression de chaque jour de la semaine courante.
- Rechargement automatique à chaque focus via `useFocusEffect`.

5.4 Écran Paramètres — SettingsScreen

Configuration complète de l'application.

- **Notifications des prières** : toggle + champ horaire pour chaque prière.
- Bouton **GPS « Localiser »** pour calculer les horaires automatiquement.
- **Mode sombre** : manuel ou automatique (déclenché au Maghrib).
- **Habitudes personnalisées** : ajout, suppression avec confirmation inline.
- **Méthode de calcul** sélectionnable : UOIF, MWL, ISNA.
- Rappels optionnels **lundi & jeudi** pour le jeûne de la Sunna.

5.5 Onboarding — OnboardingScreen

3 slides horizontales affichées uniquement au premier lancement.

- Slide 1 : **Bienvenue sur Noor** — présentation de l'application.
- Slide 2 : **Suis ta progression** — présentation des habitudes.
- Slide 3 : **Chaque jour compte** — présentation de la constance.
- Indicateurs de pagination (dots) et bouton « **Passer** » sur chaque slide.
- Dernier slide : bouton « **Commencer** » + mention ■■■ ■■■■.
- Marqueur **app:firstLaunch** en AsyncStorage pour ne plus l'afficher.

6. COMPOSANTS RÉUTILISABLES

HabitCard

```
void} />
```

Carte d'une habitude avec barre colorée à gauche, icône, texte (barré si fait), et checkbox animée. Animation spring scale 0.97 au tap.

ProgressRing

SVG pur avec deux cercles concentriques : fond gris + arc vert animé (doré si 100%). Texte centré done/total + pourcentage.

MonthCalendar

Grille 7 colonnes (Lundi en premier). Cellules colorées en 4 niveaux selon le pourcentage d'habitudes complétées ce jour-là. Bordure verte pour aujourd'hui.

Confetti

```
Déclenché automatiquement quand doneCount === total
```

36 particules animées (6 couleurs × 6 formes) : chute verticale avec dérive horizontale, rotation et fondu. Durée 2.5 secondes.

7. SYSTÈME DE THÈME DARK / LIGHT

Le thème est géré par un **Context React** (ThemeContext.tsx) qui expose isDark, colors, toggleDark() et setDarkMode(). Le choix est persisté dans AsyncStorage sous la clé **app:theme**.

Token	Light	Dark	Rôle
bg	#FAFAF8	#0D1A13	Fond principal
bg2	#F5F3EF	#112018	Fond secondaire / cartes
green	#1A6B4A	#3DAA7E	Couleur principale (accents)
gold	#B8852A	#D4A835	Couleur d'accentuation dorée
text	#1A1A18	#EDF2EE	Texte principal
textMuted	#9A9A94	#5A7A65	Texte secondaire / labels
border	#E8E6E0	#1E3828	Bordures et séparateurs

Mode sombre automatique : si AppSettings.darkModeAuto est activé, le thème bascule en mode sombre à l'heure du Maghrib. Un setTimeout est programmé pour le prochain Maghrib si l'heure n'est pas encore dépassée.

8. ALGORITHME DES HORAIRES DE PRIÈRE

Fichier : `src/utills/prayerCalc.ts`. L'algorithme est basé sur la méthode astronomique standard (PrayTimes). Il calcule les positions du soleil à partir de coordonnées GPS et de la date, sans dépendance externe.

- **Étape 1** — Calcul du Jour Julien (JD) à partir de la date (année, mois, jour).
- **Étape 2** — Position du soleil : anomalie moyenne g , longitude écliptique L , obliquité $e \rightarrow$ déclinaison solaire δ et équation du temps EqT .
- **Étape 3** — Dhuhr (midi solaire) = $12 + UTC_offset - longitude/15 - EqT$.
- **Étape 4** — Lever/Coucher : arccos avec angle -0.8333° divisé par 15.
- **Étape 5** — Fajr et Isha : $Dhuhr \pm T(angle_méthode)$. Asr : école Shafi (ombre = 1).

Méthodes supportées

Méthode	Angle Fajr	Angle Isha	Usage principal
UOIF	12°	12°	France, Maghreb (défaut)
MWL	18°	17°	Usage mondial
ISNA	15°	15°	Amérique du Nord

Exemple — Alger, 9 Juin 2026 (méthode UOIF)

Prière	Heure calculée
Fajr	04:12
Lever	05:45
Dhuhr	12:50
Asr	16:35
Maghrib	20:10
Isha	21:55

9. ALGORITHME DES STREAKS

Fichier : `src/constants/utills.ts` $\rightarrow$ fonction `calculateStreak()`. Le streak mesure la régularité de l'utilisateur : nombre de jours consécutifs où au moins une habitude a été cochée.

- Remonter en arrière à partir de **hier** (aujourd'hui est exclu).
- Pour chaque jour i (de 1 à 365) : charger les logs de la date = aujourd'hui - i .
- Si au moins un log a **done = true** $\rightarrow$ incrémenter le compteur.
- Dès qu'un jour entièrement vide est rencontré $\rightarrow$ **arrêt de la boucle**.
- Le résultat est affiché dans StatsScreen avec l'emoji ■.

Note : aujourd'hui est délibérément exclu afin de ne pas pénaliser l'utilisateur qui n'a pas encore coché ses habitudes du jour.

10. FLUX DE DONNÉES & PERSISTANCE

Toutes les données sont stockées localement via **AsyncStorage**. Aucun compte utilisateur ni connexion réseau n'est nécessaire.

Clé AsyncStorage	Type	Contenu
app:theme	string	"light" "dark"
app:firstLaunch	string	"done" — marqueur onboarding
logs:YYYY-MM-DD	JSON	HabitLog[] : { habitId, done }
habits:custom	JSON	Habit[] — habitudes ajoutées par l'utilisateur
settings:prayers	JSON	PrayerSettings — horaires et toggles de prière
settings:app	JSON	AppSettings — thème auto, méthode, rappels
notif:HABITID	string	ID de notification planifiée par habitude
notif:prayer:KEY	string	ID de notification de prière

11. COMPATIBILITÉ WEB

L'application fonctionne dans le navigateur grâce à des **stubs Metro** qui remplacent les modules natifs iOS/Android par des implémentations légères au moment du bundle web.

Module natif	Comportement web
expo-notifications	Toutes les fonctions sont des no-ops (aucune erreur silencieuse)
expo-location	getCurrentPositionAsync() → navigator.geolocation (Web API)

Un patch a également été appliqué sur **expo-modules-core** pour exposer registerWebModule(), non fourni par défaut pour la cible web.

12. LANCER LE PROJET

Prérequis

```
node >= 18 · npm >= 9
```

Installation

```
cd noor npm install
```

Commandes de lancement

Cible	Commande	Prérequis
■ Web	<code>npx expo start --web --offline</code>	Navigateur moderne
■ iOS	<code>npx expo start --ios</code>	macOS + Xcode
■ Android	<code>npx expo start --android</code>	Android Studio

En cas d'erreur Metro : vider le cache avec `npx expo start --web --offline --clear` puis `rm -rf /tmp/metro-cache`.

Aucune clé API requise. L'application est 100% locale : calcul des prières (algorithme intégré), géolocalisation (API native), stockage (AsyncStorage). Ni compte, ni connexion réseau.

ANNEXE — INTERFACES TYPESCRIPT PRINCIPALES

interface Habit

Champ	Type	Notes
id	string	Identifiant unique
name	string	"Lecture du Coran"
nameAr	string	"■■■■■ ■■■■■"
description	string	Description courte
icon	string	Nom icône Ionicons
color	string	Hex "#1A6B4A"
targetDays	number[]	[] = tous, [1,4] = lun+jeu
notificationTime	string?	"08:00"
isCustom	boolean?	true si ajouté par l'utilisateur

interface HabitLog

Champ	Type	Notes
habitId	string	Référence à Habit.id
done	boolean	true si coché ce jour

interface PrayerTimesResult

Champ	Type	Notes
fajr	string	"04:12"
sunrise	string	"05:45"
dhuhr	string	"12:50"
asr	string	"16:35"
maghrib	string	"20:10"
isha	string	"21:55"

interface Dua

Champ	Type	Notes
id	string	Identifiant unique
emoji	string	Emoji représentatif
title	string	"Dua du réveil"

titleAr	string	"■■■■ ■■■■■■■■■■"
textAr	string	Texte arabe complet
textFr	string	Traduction française
source	string	"Bukhari 6312"
category	string	"Matin" "Soir" ...

■■■ · Noor — Version 3.4.0 Aesthetic Edition · Juin 2026

Rédigé par Safia Massaid · React Native / Expo SDK 51 · 100% open source · 100% local · Aucune collecte de données